

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

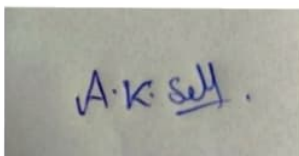
Debugging

Code of Conduct:

I A K Selva Prabhu(192421224) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with the handwritten signature "A.K. Selva" in blue ink.

Co3 AT2 Debugging

1. Debugging the Syntax-Directed Definition

$$E \rightarrow E_1 + T \{ E.val = T.val \}$$

$$E \rightarrow T \{ E.val = T.val \}$$

$$T \rightarrow \text{num} \{ T.val = \text{num.lexval} \}$$

Input : 3+5

The compiler produces Eval = 5

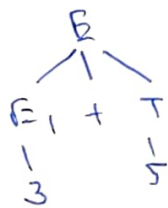
Error identification.

Incorrect semantic rule : $E \rightarrow E_1 + T \{ E.val = T.val \}$

The rule only assigns the value of T to E.

Root Cause Analysis:

3+5



$$T.val = 5$$

$$E_1.val = 3$$

$$E.val = T.val \\ = 5$$

$$E.val = E_1.val + T.val \\ = 3 + 5 \\ = 8$$

∴ The value 3 from E_1 is lost.

Corrected Semantic Rule

$$E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$$

$$E \rightarrow T \{ E.val = T.val \}$$

$$T \rightarrow \text{num} \{ T.val = \text{num.lexval} \}$$

Correct Evaluation:

$$E_1.val = 3 \\ T.val = 5$$

The semantic rule ignores the left operand.
After correcting it to
 $E.val = E_1.val + T.val$

$$E.val = 8$$

2. Debugging Syntax Tree Construction:

$a + b * c$



$$t_1 = a + b$$

$$t_2 = t_1 * c$$

Error identification:

The syntax tree is incorrect because it does not preserve the precedence of operators.

$*$ has higher precedence than $+$.

$a + b * c$

$a + (b * c)$

not: $(a + b) * c$

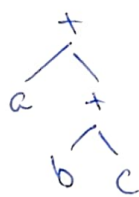
Root Cause Analysis

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$(a + b) * c$$

Corrected Syntax tree.



$*$ is evaluated before $+$.

Correct Intermediate Representation $t_1 = b * c$

$$t_2 = a + t_1$$

The final result is stored in t_2 .

Testing & Validation.

If $a = 2$
 $b = 3$
 $c = 4$

Correct expression: $2 + (3 * 4)$
 $= 2 + 12$
 $= 14$

Incorrect tree would produce:

$$(2+3)*4$$

$$= 5*4$$

The error is caused by ignoring operator precedence. The correct expression is $a + (b*c)$, with the correct intermediate code.

$$t_1 = b * c$$

$$t_2 = a + t_1$$

3). Debugging Synthesized Attributes

$$3+4*5$$

Grammar: $E \rightarrow E + T \mid T$

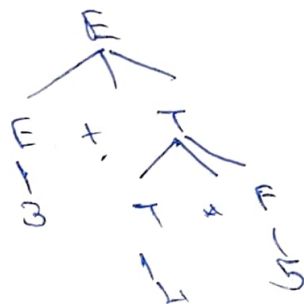
$T \rightarrow T * F \mid F$

$F \rightarrow \text{num}$

The compiler annotates the root with

$$E.val = 35$$

Attribute Evaluation



$$F.val = 3$$

$$F.val = 4$$

$$F.val = 5$$

For multiplication

$$T.val = 4 * 5$$

$$= 20$$

For add:

$$E.val = 3 + 20$$

$$= 23$$

Error Identification.

E.val = 35 is incorrect.

The value 35 would result from:

$$3 + 4 * 5 = 7 * 5 = 35$$

which incorrectly performs addition before multiplication.

Correct computation.

$$4 * 5 = 20$$

$$3 + 20 = 23$$

$$\therefore \text{E.val} = 23$$

Testing and Validation

$$3 + 4 * 5 = 3 + 20 = 23$$

$$\text{E.val} = 23$$

The incorrect computation is the root annotation E.val
The correct synthesized attributes evaluation gives:

$$\text{B.val} = 23$$

Debugging the type-checking process

Identifies

Type

a

int

b

float

c

int

*

float

Statement

$$x = a + b * c$$

Febr mid,

$c \rightarrow \text{float}$

$b * c \rightarrow \text{float}$

Type Error: incompatible operands

Step 1: Analyze $b * c$

$b \rightarrow \text{float}$

$c \rightarrow \text{int}$

Step 2 Analyze $a + (b * c)$

$a \rightarrow \text{int}$

$b * c \rightarrow \text{float}$

Step 2: ~~Assign~~ $a \rightarrow \text{float}$

$\therefore a + (b * c) \rightarrow \text{float}$

Step 3: Check Assignment

$x \rightarrow \text{float}$

$a + b * c \rightarrow \text{float}$

$\text{float} \rightarrow \text{float}$

No final conversion is required.

Const type-checking Process

$t_1 = \text{float}(c)$

$t_2 = b * t_1$

$t_3 = \text{float}(a)$

$t_4 = t_3 + t_2$

$x = t_4$

Type Error: incompatible operands

$b * c \rightarrow \text{float}$

$a + (b * c) \rightarrow \text{float}$

$x \rightarrow \text{float}$

The assignment is valid after the required integer-to-float conversion

Debugging Type checking and Conversions:

```
int a;  
float b;  
int x;
```

statements:

```
x = a + b ;
```

```
b = a + x;
```

both valid. but does not generate any conversion

Statement 1: $x = a + b$

Types: $a \rightarrow \text{int}$

$b \rightarrow \text{float}$

~~@~~ ~~$\rightarrow$~~ before add.

$a \rightarrow \text{float}$

$a + b \rightarrow \text{float}$

$x \rightarrow \text{int}$

$\text{float} \rightarrow \text{int}$

Correct Processing

$t_1 = \text{float}(a)$

$t_2 = t_1 + b$

$x = \text{int}(t_2)$

eg: $x = (\text{int})(a + b);$

Statement 2: $b = a + x$

Types: $a \rightarrow \text{int}$

$x \rightarrow \text{int}$

$a + x \rightarrow \text{int}$

$b \rightarrow \text{float}$

$t_1 = a + x$

$t_2 = \text{float}(t_1)$

$b = t_2$

Error identification

Statement

$x = a + b$

$b = a + x$

Expression Type

float
int

Variable

int
float

Required Conversion

float $\rightarrow$ int

Answer:

$x = a + b;$

$a + b$ produces a float, so assigning it to integer x requires a narrowing conversion. It may lose the decimal

for $b = a + x;$

$\therefore$ The type checker should generate the required conversion. //